IDEA Field Support over Telegram

Near-real-time chat between the field and the Support Engineer

Proposal · Status: Draft for Koen

IDEA - offline school computing. Prepared for Koen Swings's team.
No real passwords, keys or tokens are included; values in <ANGLE BRACKETS> are placeholders.

Contents

What and why 3
Research findings (20 October 2026) 4
Approach options 5
Cost 6
Security 7
Approval in phase 0 8
Setup 9
Open questions for Koen 10

Author: Steve (Lead Bot) **Date:** 2026-10-20 **Status:** Draft. Extends the approved field support channel design (idea#188).

What and why

Today Justice writes to `idea-support@agentmail.to`. Each round trip is an email, picked up by a watcher at most once a minute, then answered by email. That works, but a back-and-forth conversation ("what does the screen show now?") takes many slow rounds.

Proposal: add a private **Telegram bot** (for example `@IdeaFieldSupportBot`) as the main field channel. Justice chats with it on his phone like any other chat. Each message reaches the Support Engineer within seconds and the answer comes back in the same chat. Email stays as the fallback.

Why Telegram rather than WhatsApp (phase 1 in the earlier design): no business verification, no phone number to buy, no 24-hour window or templates, no per-message fees, and a bot can be created in five minutes. Telegram also works well on weak mobile connections.

Affected domains: field support (Support Engineer, Marco, Steve), the box (one small relay script), the routine "Box watcher events". No change to the Engine, Console, apps, client cards or routers.

Research findings (20 October 2026)

- **No native Telegram channel in Grok Bot.** There is no Telegram connector or tool in the available integrations, and the app shows a Channels section only when a channel connector exists. Grok Bot's own native inbound channel is email (`mail.grokbot.com`).
- **Telegram Bot API** is free. A bot receives messages either by **long polling** (`getUpdates`, the request waits up to 50 s and returns immediately when a message arrives) or by a webhook. It sends with `sendMessage`. No AI is needed for either.
- **Bot chats are not end-to-end encrypted.** They are cloud chats stored on Telegram's servers. Fine for support questions, not for secrets.

Approach options

Option	How	Speed	AI cost	Verdict
A. Relay script on the box	Script long-polls Telegram; for each allowed message it calls the existing "Box watcher events" webhook (`type: telegram_message`); the bot answers by running a small `tg-send` command on the box, which calls `sendMessage`	2 to 10 s to wake, plus answer time	Only per real message	**Recommended**
B. Telegram MCP connector	Third-party MCP server for Telegram	Polling-based, no instant wake	Each check is an AI run	Not recommended: no trusted connector, costs on empty checks
C. Telegram to email bridge	Forward Telegram messages into the support inbox	Same minute delay as email	Per message	Gains little over email

Option A in detail

1. `tg-relay` (systemd service on the box, about 100 lines of Python): long-polls `getUpdates`, drops anything not from the allowlist, stores each message (text, photos as files) under `/workspace/support/telegram/`, and calls the webhook once per message, batching messages that arrive within a few seconds.
2. The webhook routine wakes the Support Engineer with the chat id and message. It answers by running `tg-send <chat> <text|photo>`. Replies are not generated for empty polls; the script polls, not the AI.
3. Night rule: Koen's 07:00 to 24:00 rule applies to messages to Koen, not to Justice. Field messages are answered at any hour (Zimbabwe is UTC+2: the same as Belgium until 25 October, one hour ahead after that).
4. Logging: every in and out message is appended to the support log, as with email, so Steve can report and nothing gets lost.

Cost

- Telegram: free. Relay script: no AI, runs on the box.
- AI runs: one per real incoming message (or burst), the same as the email channel today. A conversation of 10 messages costs about 10 short runs, roughly the same as 10 emails, but finishes in minutes instead of an hour.
- Once Telegram works, the email watcher can stay (it is free) and the old 30-runs-a-day timer routine is switched off.

Security

- **Allowlist:** only Koen's and Justice's Telegram user ids (numbers, not usernames). The bot ignores everyone else and logs the attempt. Adding a school later means adding an id.
- **Bot token** is stored on the box only, set through a secret request, never printed or put in chat or Git.
- **No passwords in chat**, either way. If a password is needed, the bot points to the credentials sheet Justice already has.
- **No system access:** same rule as the approved design. The Support Engineer filters and explains; it never deploys, changes configuration or uses SSH writes. Anything else becomes a report to Steve and Koen.
- **Data:** Telegram stores bot chats in its cloud (not end-to-end encrypted). Do not send school pupil data or credentials through it.

Approval in phase 0

Phase 0 is "draft only, Koen approves". Real-time chat needs a lighter rule. Proposed:

- **Answer directly** when the answer comes from merged docs (manuals, tutorial, remote-access guide, known issues): "Which cable goes where", "how do I reflash a card". The bot names the source section.
- **Ask clarifying questions directly** (sticker name, what the screen shows, a photo).
- **Escalate to Koen** everything else: anything that may need a change, a judgement call, a promise, or is not covered by the docs. The bot tells Justice "I've asked Koen, I'll come back to you", and Koen gets the draft answer to approve.

Setup

Koen (about 10 minutes):

1. In Telegram, open `@BotFather`, send `/newbot`, choose a name and username.
2. Paste the token into the secret request Steve sends (not into chat).
3. Send the bot one message, so it learns your user id.
4. Send Justice the bot link (`https://t.me/<username>`) and ask him to press Start and send "hello".

Bots:

- Steve: write and install `tg-relay` and `tg-send`, add `telegram_message` to the "Box watcher events" routine, record the two user ids in the allowlist, test with Koen's account, then with Justice.
- Support Engineer and Marco: answer through `tg-send`, following the approval rule above.
- Atlas: watch that the relay service is running (part of the box watchers).

Open questions for Koen

1. Approval rule: OK to answer directly from merged docs and escalate everything else, or keep "draft only, you approve" for every reply?
2. Bot name and username (for example "IDEA Field Support", `@IdeaFieldSupportBot`)?
3. Keep email as the fallback channel, or retire it once Telegram works?
4. Does Justice have Telegram and enough mobile data in Zimbabwe? Should the bot also accept voice notes (transcribed) and photos? Photos are proposed yes, voice notes later.
5. Should you also be in the chat (a small group with Justice and the bot), so you can see conversations live, or only get escalations?